

Role-Play App

Local, fully offline AI role-play chat application — three reference sheets covering component architecture, the chat-turn data flow, and the SQLite data model.

INFERENCE

llama-server · Vulkan · 1 slot

STORAGE

SQLite + sqlite-vec

BACKEND

FastAPI · uvicorn :8000

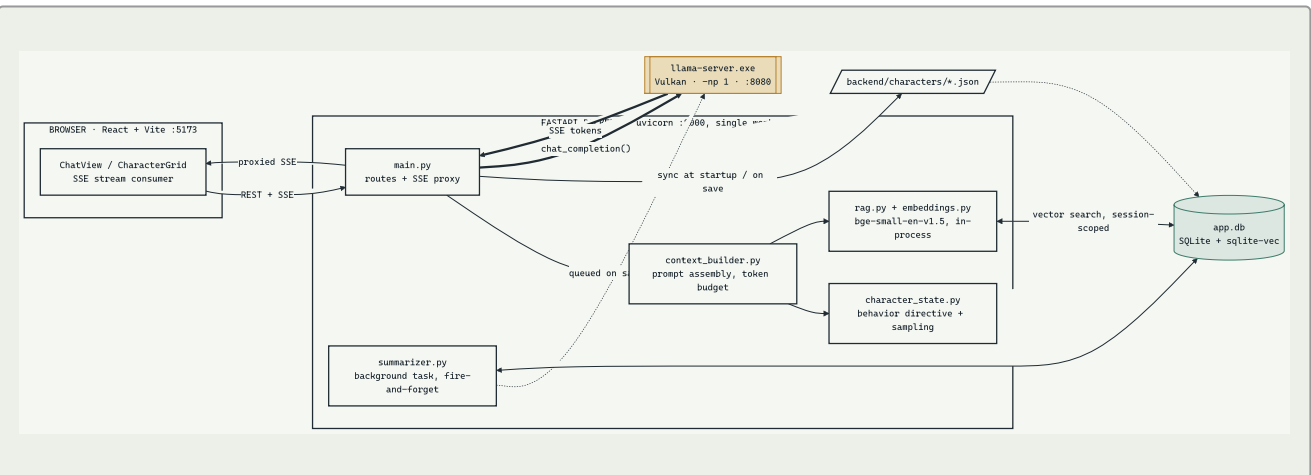
FRONTEND

React · Vite :5173

SHEET 01

Component Architecture

Everything runs on one machine, no external services. The backend brokers between the browser, one llama-server inference slot, an in-process embedding model, and a single SQLite file.



— single inference slot — chat & summarizer both queue here

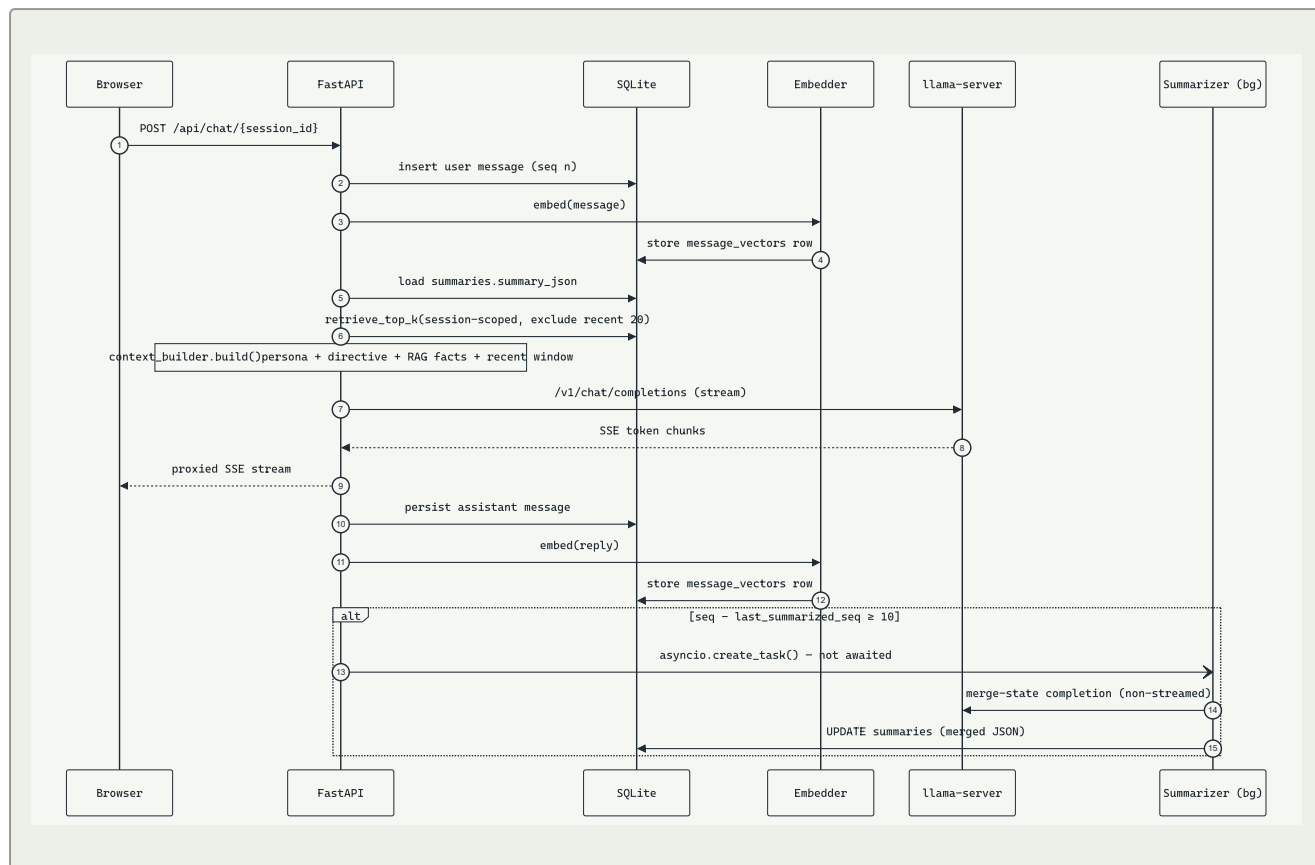
— only data store — one SQLite file, no external services - - - disk → DB sync (character cards)

SHEET 02

Chat-Turn Data Flow

One user message. start to finish. The reply streams back before the background

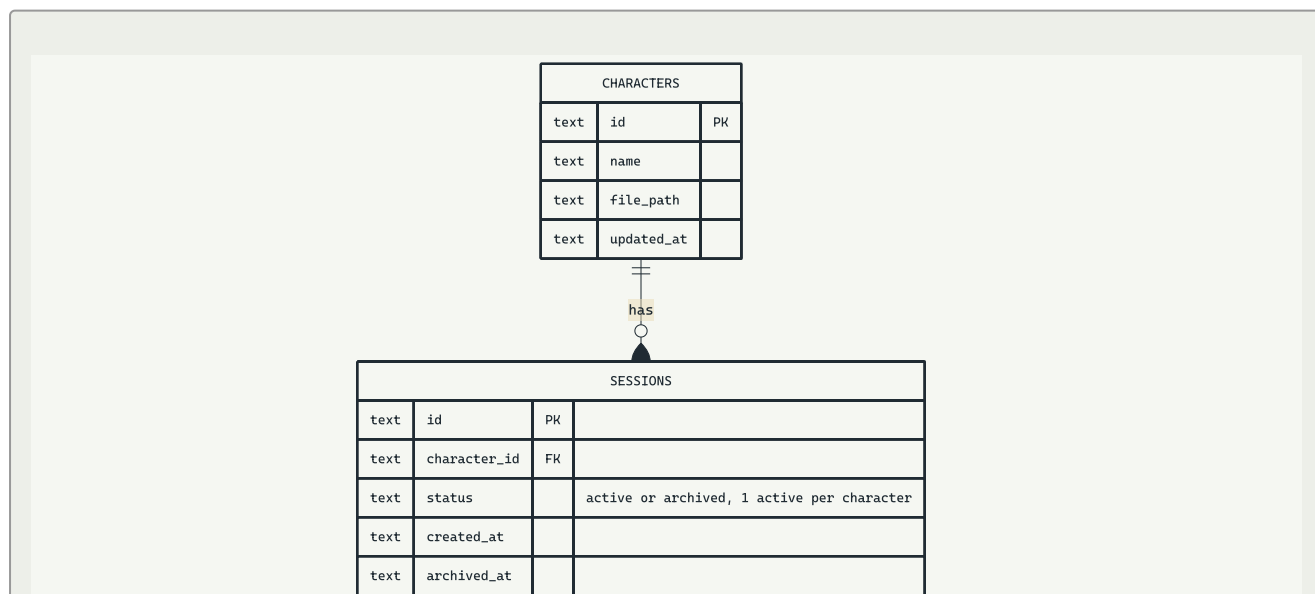
summarizer is ever triggered — the user's turn is never delayed by it.

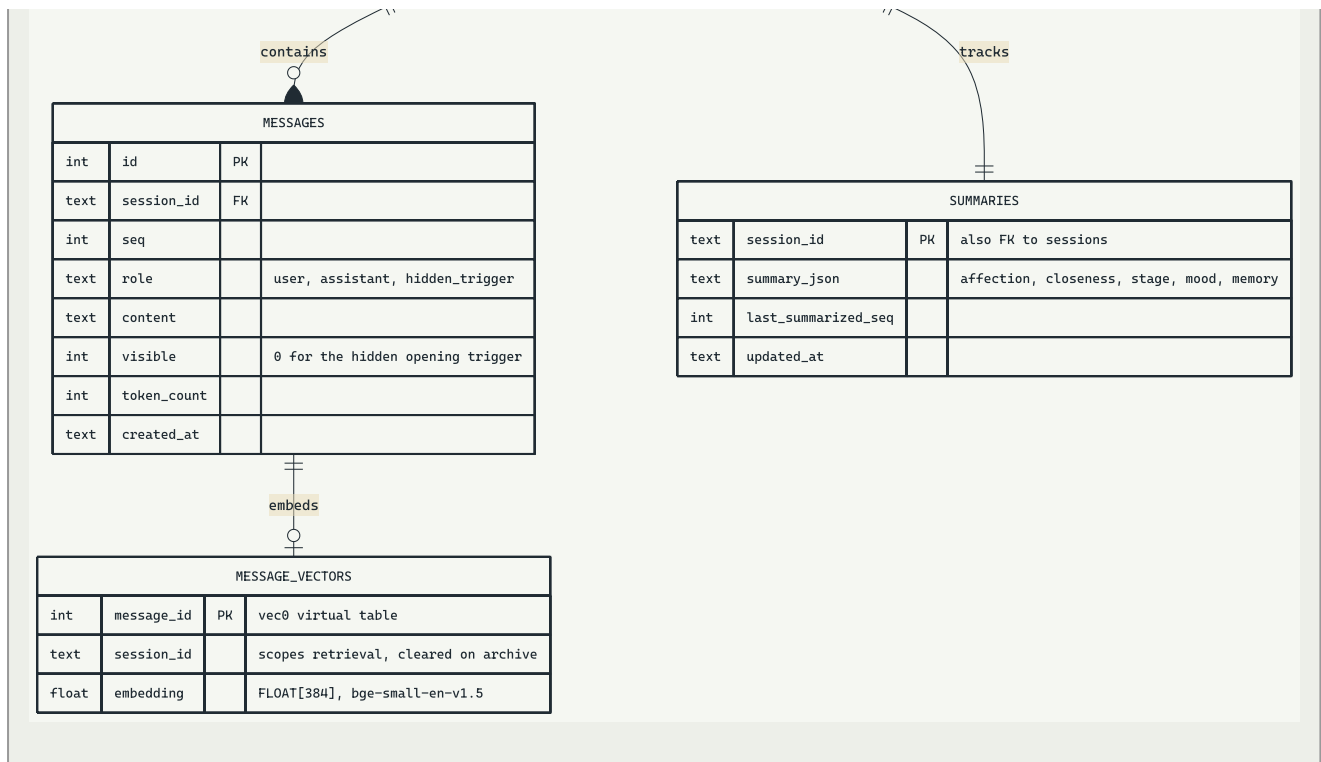


SHEET 03

Data Model

One row per session in **summaries** holds the character's evolving state — affection, closeness, relationship stage, mood, memory — merged, not overwritten, on every summarizer pass.





NOTES

Load-Bearing Decisions

Choices that shape the diagrams above and aren't obvious from the boxes alone.

Single inference slot

Chat and background summarization share one llama-server slot (`-np 1`) — deliberate, given tight RAM headroom. Costs latency, never correctness.

Session-scoped RAG

Vector search never crosses sessions. Clear-chat archives the old session; its facts stop being recalled from the moment a new one starts.

Merge, never overwrite

Every summarizer pass merges into `summary_json`. A bad or partial model output can't silently erase established character memory.

Single uvicorn worker

The summarizer's in-flight guard is process-local memory. A second worker would double-run summaries with no shared lock — the app requires exactly one.

